HGKT: Hypergraph-based Knowledge Tracing for Learner Performance Prediction

Yuwei Ye
School of Computer Science
South China Normal University
Guangzhou, China
yeyuwei@m.scnu.edu.cn

Zhilong Shan*
School of Computer Science,
School of Artificial Intelligence
South China Normal University
Guangzhou, China
zlshan@m.scnu.edu.cn

2023 International Joint Conference on Neural Networks (IJCNN) | 978-1-6654-8867-9/23/$31.00 ©2023 IEEE | DOI: 10.1109/IJCNN54540.2023.10191844

***Abstract*—Knowledge tracing focuses on modeling learners' past answer sequences to trace the evolving knowledge state and predict their performance in the future. Most of the existing GNN-based knowledge tracing model only considers the static pairwise relationship between concepts and exercises, but ignores the mining of edge features. Also, the dynamic and complex higher-order relationships hidden in the learners' answer sequence have not been fully exploited. In this paper, a novel hypergraph-based knowledge tracing model (HGKT) is proposed to address these limitations. Firstly, we exploit edge feature that indicates the frequency of exercise-concept's occurrence to extend the common bipartite graph. Then we use Node and Edge features based graph Neural Networks (NENN) to obtain the embedding representation of exercises and concepts. Secondly, a hypergraph with different weights on vertices is constructed during the learners' exercise-answering process and then it is transformed to a simple graph based on its similarity between hyperedges. Thereafter, we use the hypergraph neural networks (HGNN) and line hypergraph convolution network (LHCN) to obtain the learners' embedding and discover the higher-order relationships formed during this process. Thirdly, the difficulty of exercises and the average response time are utilized to improve the learning of LSTM's hidden states. Finally, all the embeddings are jointly added to the generalized interaction module of GIKT to draw attention to the useful information for prediction. Experiments demonstrate that the proposed HGKT outperforms previous classical methods in terms of AUC on the three widely used datasets.**

***Index Terms*—Knowledge Tracing, Hypergraph Neural Networks, Line Hypergraph Convolution Network, Learner Representation, Attention Mechanism**

I. Introduction

Knowledge tracing (KT) is an essential and challenging fundamental task in computer-assisted Intelligent Tutoring Systems (ITS). It is used to trace the evolving knowledge state of learners over time and predict how well they will perform in the future. Especially in the Internet era where the COVID-19 epidemic has emerged, the way of learning for students has shifted dramatically from offline classroom to online learning [1], where students consolidate their knowledge gains by completing post-class exercises. With the aid of computers, teachers can assess students' performance through their responses, offer accurate and personalized diagnosis and instructions, and recommend personalized learning paths that match students' preferences and knowledge levels [2], thereby alleviating teachers' workload [3] and enhancing the learning effectiveness of students.

With the flourishing of artificial intelligence technology in recent years, deep learning methods have been introduced into the field of education. The groundbreaking Deep Knowledge Tracing (DKT) model [4] was proposed in 2015, which introduces the variant of Recurrent Neural Networks (RNNs), namely LSTM [5], to model the learners' exercise-answering sequence. It pioneered the application of deep learning for knowledge tracing and achieved a better performance than traditional machine learning based methods. And LSTM is designed with a high-performance neural network and excellent modeling capacity for sequential modeling tasks and has been widely used in various sequence-based models to solve different problems in knowledge tracing, such as the double-sequence RNN on the programming exercises [6], the bidirectional LSTM based EKT [7], the DKT-based model combined with multiple features about forgetting for hidden state learning [8]. From the above discussion, it is clear that RNNs are well suited for modeling sequential data.

Later on, as graph representation learning made a big breakthrough in the field of deep learning, Nakagawa et al. introduced Graph Neural Networks (GNN) to the field of knowledge tracing for the first time in 2019, named GKT [9]. GKT adopts a directed graph to model knowledge structure of concepts for the first time to trace learners' knowledge states and transforms the knowledge tracing task into a time-series node-level classification problem in the GNN domain without considering the relationship between exercises and concepts. To model exercise-concept relations, PEBG [10] is the first to pre-train the question embedding on a bipartite graph with useful side information like difficulty and can be integrated with the existing KT models to improve performance. Moreover, Yang et al. proposed a Graph-based Convolutional Neural Networks (GCNs) for knowledge tracing in 2020, named GIKT [11], where GCNs perform convolutional operations on nodes of the graph. It can be used to aggregate correlations on a static e2c bipartite graph that contains neighborhood information and high-order structure information. However, they all inadequately utilize the edge features that contain important information to obtain a better node embedding. It is necessary to take both the node and edge features into account

Authorized licensed use limited to: HUNAN UNIVERSITY. Downloaded on December 04,2023 at 10:41:15 UTC from IEEE Xplore. Restrictions apply.

while updating node representations in graph components [12]. And since different neighbors play a different role in generating the node embedding, we use the frequency of e2c occurrence as edge weight and use the attention-based NENN [13] for this weighted bipartite graph. To be specific, if an exercise or concept showed up more often, we are more likely to master it to some extent, which is consistent with human cognitive learning.

Besides, from a student learning perspective, these models ignore the dynamic higher-order relationships and the changing skill mastery among students during the exercise-answering process. Inspired by hypergraph theory [14], we believe that the hypergraph can be flexibly and efficiently used to model this process, where its hyperedge can connect more than two vertices. And the hypergraph convolution layer can efficiently extract both low-order and high-order correlation on hypergraph by the vertex-hyperedge-vertex transformation compared with graph convolution [15]. Learners do a variety of post-lesson exercises during the learning process, which means they interact with the exercises and skills, and hypergraph is naturally tailored to model such an interaction scenario and capture higher-order relations.

To better illustrate this thinking, a toy example is shown in Fig. 1. On the upper part, there are three students doing exercises and suppose they all encounter the exercise e_4 which contains certain knowledge concepts at time step t. The task is to predict whether they will answer it correctly. When using hypergraph, we can model the explicit and implicit relations among them as shown on the lower part of Fig. 1. This hypergraph is constructed with exercises as vertex and learners as hyperedge during their exercise-answering process. Then it will be transformed to a simple graph with learner as node and similarity as edge weight. Because learner A has already encountered exercise e_4 before, and learners A and C share two of the same nodes indicating a greater similarity. We can tell that learners A and C could have a larger probability of answering exercise e_4 correctly and a better mastery than learner B. No matter whether the exercise is answered correctly or incorrectly, the learner still has gains in skill mastery [16]. And exercises in hyperedge should have a different importance, so we use an attention mechanism [17] that assigns different weight to incidence matrix for measuring the importance of each exercise node in hypergraph, i.e., exercises that have been answered multiple times will have a greater chance of being mastered, and likewise the probability of answering exercises containing the same concept correctly will increase.

To sum up, the main contributions of our work are as follows:

- We propose a novel graph-based knowledge tracing model using a hypergraph to model dynamic and high-order relations formed in exercise-answering sequences, followed by mapping the hypergraph into a line hypergraph for learner embedding.
- This study exploits the edge feature that indicates the frequency of exercise-concept's occurrence to extend the exercise-concept bipartite graph.
- To improve the representation learning of LSTM's hidden states, the difficulty of exercise and the average response time are integrated into exercise embedding.
- Experiments conducted on three public datasets demonstrate that our proposed HGKT is superior to some of the classic and representative knowledge tracing models.

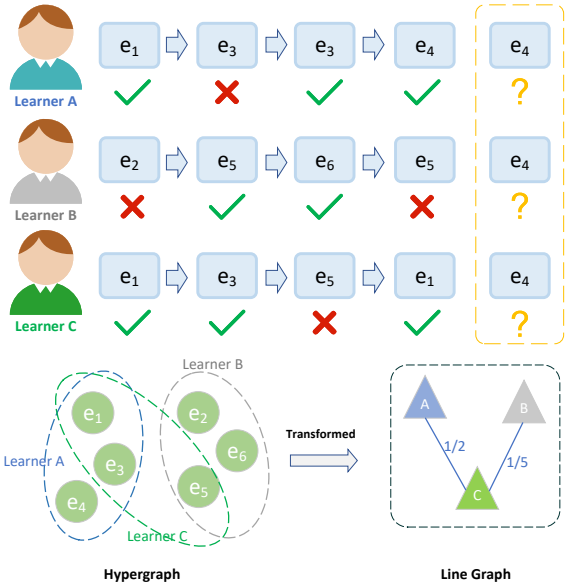


Fig. 1: Example of constructing a hypergraph during the learning process at time step t.

II. Related Work

In this section, we first review the prior related research on graph neural networks and knowledge tracing. Then, the related work on graph-based knowledge tracing is summarized.

A. Graph Neural Networks

With the rapid development of deep learning in recent years, graph-based neural networks have recently emerged and demonstrated superior performance on a variety of problems in a wide range of domains [18]. Gori et al [19] first proposed a graph neural networks (GNNs) model to process data with graph topology. Then in 2016, graph convolutional neural networks (GCNs) [20] was proposed as a model for semi-supervised classification tasks based on the spectral domain. The convolution operation defined on a graph can sample and aggregate information from neighbors to update the representation of the target nodes. In addition, there are variants of the model adapted to different scenarios according to different improvement methods, such as GraphSAGE [21] and LightGCN [22].

A hypergraph is a special kind of simple graph that extends the theory that each edge in a simple graph can only connect two nodes to the theory that each edge can connect multiple nodes at the same time. When each hyperedge is connected to only two nodes, it becomes a simple graph. It

Authorized licensed use limited to: HUNAN UNIVERSITY. Downloaded on December 04,2023 at 10:41:15 UTC from IEEE Xplore. Restrictions apply.

has attracted widespread interest due to its inherent capabilities and flexibility in modeling complex high-order correlations, which is more superior than simple graph [23]. Zhou et al. [24] first proposed the concept of hypergraph learning, applied clustering methods based on spectral decomposition to hypergraph, and proposed a hypergraph embedding algorithm. In 2019, Feng et al. proposed a hypergraph neural network (HGNN) [15] based on spectral decomposition and designed a hyperedge convolution operator. Later on, they proposed a general hypergraph neural network, named HGNN+ [25], which extends HGNN to handle high-order multi-modal/multi-type data correlation more efficiently by employing an adaptive hyperedge group fusion strategy [25]. Hyperedges that share at least one node are considered to be related. So Bandy et al. proposed a line hypergraph convolutional network (LHCN) [26] that constructs an undirected simple graph by treating each hyperedge as a node, allowing simple graph convolutional neural networks to be applied to hypergraph structures.

B. Graph-based Knowledge Tracing

Researchers in this field have made many contributions to the task of knowledge tracing, and previous work can be divided into two categories in terms of modeling techniques: traditional machine learning methods and deep learning methods. Recently, due to the great capacity and effectiveness of graph representation learning, more and more KT methods utilize graph to model complex relationships and interdependencies between objects. So this subsection will be focused on the graph-based deep knowledge tracing model. Aside from the classic graph-based KT models previously mentioned, here we will introduce the most recent ones such as HiGKT [27], SKT [28], JKT [29], Bi-CLKT [30], and SGKT [31].

HiGKT introduces a hierarchical graph with problem schema to explore the latent hierarchical relations and dependencies between exercises. It explores the rich information contained in exercises but ignores knowledge concepts. JKT tries to exploit the multi-dimensional exercise-exercise and concept-concept relationships into graph and fuse them with the exercise-concept relationship based on a joint graph convolution network, which makes it possible to discover correlations between exercises under cross-concepts and help obtain high-order semantic information. SKT focuses on exploiting the multiple relations in knowledge structure, such as similarity relations and prerequisite relations, to model the propagated influence both temporally and spatially among concepts. Bi-CLKT integrates node-level and graph-level contrastive learning to obtain the representations of exercises and concepts, respectively, on an e2e and c2c subgraph in a self-supervised manner. And SGKT uses a directed dynamic session graph to model the exercise answering transition and a heterogeneous tripartite graph to model the high-order correlations among user-exercise-concept.

III. Preliminarilies

In this section, we first formally denote the main goal of knowledge tracing and then present the definitions of the weighted bipartite graph, the learning interaction hypergraph and the line hypergraph.

A. Task of Learner Performance Prediction

The primary task of knowledge tracing is to model learners' changing knowledge states over time based on historical records of exercise-solving in ITS and to predict learners' future performance. Mathematically, this can be defined as a time series prediction task: given an exercise-answering sequence of each learner from time step 1 to t: $X_t = \{(e_1, a_1), (e_2, a_2), \ldots, (e_t, a_t)\}$, in which tuple (e_t, a_t) represents $(exercise, correct\ or\ not)$ at time step t. The ultimate goal is to predict the probability that this learner will answer correctly when encountering a new exercise e_{t+1} at the next time step $t+1$: $P_{t+1}(a_{t+1} = 1 \mid X_t, e_{t+1})$.

B. Weighted Bipartite Graph

An exercise may contain one or multiple knowledge concepts, and a knowledge concept may be related to multiple exercises. We follow and extend [11] to define a heterogeneous undirected weighted exercise-concept bipartite graph as $WBG = (E, R, C, W)$, where $E = [e_1, ..., e_n]$ is the set of exercises, $S = [c_1, ..., c_m]$ is the set of knowledge concepts, in which n and m represent the number of exercises and concepts, respectively. $R = [r_{ij}] \in \{0, 1\}^{|E| \times |C|}$ is a binary adjacency matrix and W is the matrix of edges weight, the entries of which are the normalized values of edges weight from 0 to 1, with 0 indicating no edge between nodes but 1 indicating a strong connection.

C. Learning Interaction Hypergraph

Traditional graph convolutional neural networks are typically employed for simple graph $G = (V, E)$ but incapable of capturing complex and high-order relations in the hypergraph. When hypergraph is integrated into knowledge tracing, learners are treated as hyperedges and exercises as nodes, which can be used to model the relationship formed during the exercise answering process. Exercises answered by the same person are enclosed together with a hyperedge. Algebraically, we denote a hypergraph as $HG = (V, H, E, W)$, where $V = [v_1, ..., v_n] = [e_1, ..., e_n]$ represents the set of exercise vertices, $E = [\epsilon_1, ..., \epsilon_k] = [l_1, ..., l_k]$ represents the set of learner hyperedges, and W is a diagonal matrix representing the weight of the hyperedges. Each hyperedge can be assigned a weight value based on its importance, but in this case we set them all to 1. The hypergraph structure is often represented by a $|V| \times |E|$ incidence matrix H which is used in HGNN to guide each vertex to aggregate and generate the hyperedge feature. We follow [25] to construct hypergraph with normalized edge-dependent vertex weights. A greater weight indicates that the node has more importance in that related hyperedge. If $v_i \in V$ and $\epsilon_j \in E$, the entries of H are denoted as

$$H_{ij} = \begin{cases} x \in (0, 1], & \text{if } v_i \in \epsilon_j \\ 0, & \text{if } v_i \notin \epsilon_j \end{cases} \tag{1}$$

Authorized licensed use limited to: HUNAN UNIVERSITY. Downloaded on December 04,2023 at 10:41:15 UTC from IEEE Xplore. Restrictions apply.

Furthermore, the degrees of nodes and hyperedges are defined by

$$D_i = \sum_{j=1}^{n} W_j H_{ij} \tag{2}$$

$$B_j = \sum_{i=1}^{k} H_{ij} \tag{3}$$

D. Line Graph of Hypergraph

As previously mentioned, it is clear that learners with similar answer processes have similar and relevant ability. The learning interaction hypergraph (HG) is constructed in the previous subsection and then we transform it into a line graph, named LHG, with weights on edges in order to obtain better representation of the relationships among learners as well as to optimize the learner embedding representation. LHG is a simple graph with the hyperedges as nodes and the correlation between the hyperedges as edges, so that we can employ GCNs on it.

The method proposed in [26] is able to transform the hypergraph into a line graph by treating different-sized hyperedges as distinct nodes in simple graph. If two hyperedges share a subset of common vertexs, they will be connected by an edge in simple graph. While the edge weight of the line graph LHG is determined by the number of shared nodes of the hyperedges divided by the total number of nodes in this two hyperedges. Formally, given a hypergraph $HG(V, H, E, W)$, then following [26], a line graph of hypergraph can be denoted as $LHG(V_L, E_L, W_L)$ where $V_L = \{\mathbf{v}_\epsilon \mid \epsilon \in E\}$, $E_L = \{\{\mathbf{v}_{\epsilon_p}, \mathbf{v}_{\epsilon_q}\} \mid |\epsilon_p \cap \epsilon_q| \geq 1; \epsilon_p, \epsilon_q \in E\}$ and $W_L = w_{p,q}$. And $|V_L|$ is the number of learners and $|E_L|$ is the number of pairwise learners. A learner hyperedge is considered as a new node in the line graph. If two hyperedges share at least one common exercise node, they will be connected as new nodes in the line graph. Then the edge between these two nodes p and q will be assigned a weight $w_{p,q}$ to measure the similarity of these two hyperedges as follows:

$$w_{p,q} = \frac{|e_p \cap e_q|}{|e_p \cup e_q|} \tag{4}$$

IV. Methodology

This section presents the HGKT framework proposed in this paper. As depicted in Fig. 2, the overall model architecture consists of three parts: the graph embedding module, the sequence modeling module, and the information interaction prediction module.

The embedding module is comprised of the representation learning of WBG using a Node and Edge features incorparated graph Neural Networks (NENN) and HG using a general two-stage hypergraph neural networks (HGNN) and line hypergraph convolution network (LHCN); the sequence modeling module outputs students' evolving knowledge states with a more enhanced input considering the difficulty and the average response time; and the information interaction prediction module incorporates critical information from the previous module to improve prediction performance.

A. Exercise and Concept Embedding Module

In this module, we use an attention mechanism based NENN [13] to obtain the embedding representation of exercises and concepts from WBG. We can still access explicit exercise-concept relations, implicit question similarity and concept similarity the way model PEBG [10] did. In addition, we can also obtain the embedding containing more semantic information with the edge feature.

Before going further, we statistically analyze the frequency of exercise-concept occurrence from the training set of the dataset as the edge weights on the foundation of the previous unweighted bipartite graph. To better expolit the semantic information of edge feature in vertices, we then use a node-level attention to learn the importance coefficients of node-based neighbors. Suppose the input features consist of two kinds of node features: exercise feature matrix $\mathbf{E} = [\mathbf{e_1}, ..., \mathbf{e_n}]$ and concept feature matrix $\mathbf{C} = [\mathbf{c_1}, ..., \mathbf{c_m}]$, and edge feature matrix $\mathbf{E}' = [\mathbf{e}'_\mathbf{1}, ..., \mathbf{e}'_\mathbf{k}]$. Node features are initialized randomly, and scalar edge weights are preprocessed to $[0, 1]$ by Min-Max normalization [32] to avoid increasing the scale of output features through propagation and then mapped to the 100-dimensional embeddings using a fully connected layer:

$$\mathbf{E}'_{norm} = W_e \left(\frac{\mathbf{E}' - \mathbf{E}'_{min}}{\mathbf{E}'_{max} - \mathbf{E}'_{min}} \right) + b_e \tag{5}$$

Then the importance of node j or edge k to node i can be denoted as follows:

$$\epsilon^n_{ij} = Att^n_{node}\left(\mathbf{W}_n x_i, \mathbf{W}_n x_j\right) \tag{6}$$

$$\epsilon^e_{ik} = Att^e_{node}\left(\mathbf{W}_n x_i, \mathbf{W}_e e'_k\right) \tag{7}$$

For our purpose, $\mathbf{X} = [\mathbf{x_1}, ..., \mathbf{x_{n+m}}] = Concat(\mathbf{E}, \mathbf{C})$ which concatenates the $\mathbf{E}$ and $\mathbf{C}$ at zero-dimension. The above Att^n_{node} and Att^e_{node} put the matrix to a fully connected network which performs node-level attention for node i. W_n and W_e are the learnable weight matrices that linearly transform the input features into high-level space. Next, the importance coefficients α^n_{ij} of node j to node i and α^e_{ik} of node i and edge k can be calculated as follows:

$$\alpha^n_{ij} = \frac{\exp\left(\sigma\left(a_n^T[W_n\mathbf{x}_i, W_n\mathbf{x}_j]\right)\right)}{\sum_{k\in\mathcal{N}_i}\exp(\sigma(a_n^T[W_n\mathbf{x}_i, W_n\mathbf{x}_k]))} \tag{8}$$

$$\alpha^e_{ik} = \frac{\exp\left(\sigma\left(a_e^T\left[W_n\mathbf{x}_i, W_e\mathbf{e}'_k\right]\right)\right)}{\sum_{j\in\mathcal{E}_i}\exp\left(\sigma\left(a_e^T\left[W_n\mathbf{x}_i, W_e\mathbf{e}'_j\right]\right)\right)} \tag{9}$$

where $[,]$ is the concatenation operation, N_i represents the neighbor nodes of node i and ϵ_i represents the neighbor hyperedges of node i, a_n and a_e are the parameter vectors of a single-layer fully connected network and σ is the activation function LeakyReLU.

Finally, after integrating the structure information into NENN and calculating the importance coefficient, we can get the aggregated embedding representation of vertex i in l-th layer:

$$\mathbf{x}_{\mathcal{N}_i} = \sigma\left(W_n \cdot \text{MEAN}\left(\left\{\alpha^n_{ij}\mathbf{x}_j, \forall_j \in \mathcal{N}_i\right\}\right)\right) \tag{10}$$

$$\mathbf{x}_{\mathcal{E}_i} = \sigma\left(W_e \cdot \text{MEAN}\left(\{\alpha^e_{ik}\mathbf{e}_k, \forall_k \in \mathcal{E}_i\}\right)\right) \tag{11}$$

Authorized licensed use limited to: HUNAN UNIVERSITY. Downloaded on December 04,2023 at 10:41:15 UTC from IEEE Xplore. Restrictions apply.

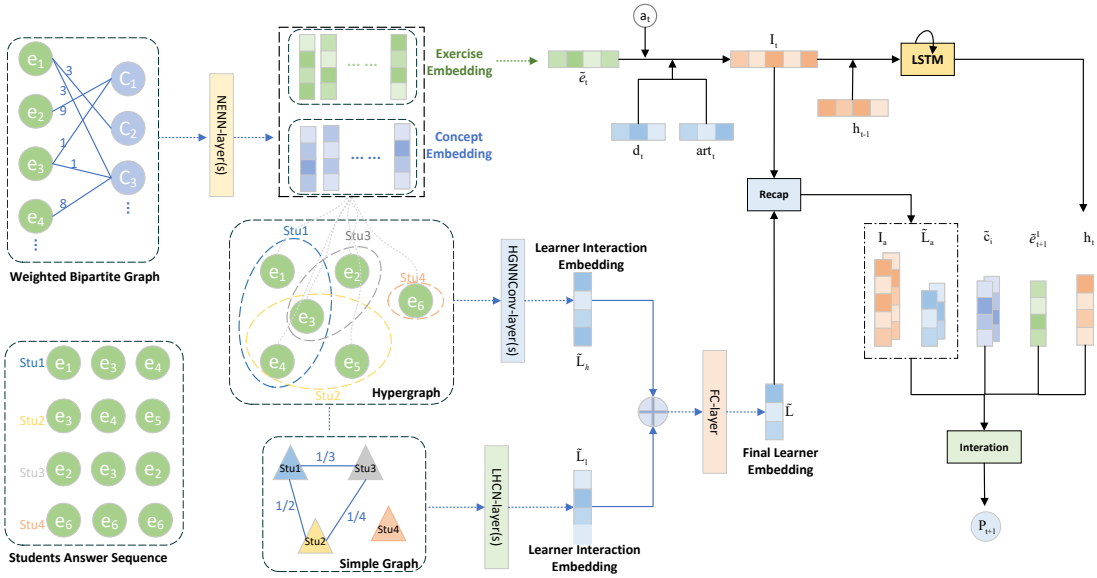


Fig. 2: The architecture of HGKT.

$$\mathbf{x}_i^{(l)} = \text{CONCAT}\left(\mathbf{x}_{\mathcal{N}_i}^{(l-1)}, \mathbf{x}_{\mathcal{E}_i}^{(l-1)}\right) \tag{12}$$

where MEAN is a mean aggregator, $x_{\mathcal{N}_i}$ and x_{ϵ_i} are node i's node based and edge based neighbors aggregated embedding with the corresponding importance coefficients respectively. We use $\widetilde{\mathbf{E}} = [\widetilde{\mathbf{e}}_\mathbf{1}, ..., \widetilde{\mathbf{e}}_\mathbf{n}]$ and $\widetilde{\mathbf{C}} = [\widetilde{\mathbf{c}}_\mathbf{1}, ..., \widetilde{\mathbf{c}}_\mathbf{m}]$ to represent the exercises and concepts representation after embedding propagation through NENN's multiple layers of aggregator.

B. Learning Interaction Embedding Module

This module first introduces HGNN based representation learning for HG, where the incidence matrix uses attention to represent the importance of the exercise nodes in the hypergraph. The hypergraph is then transformed into a line hypergraph where LHCN is used to obtain the representation of learners' node embedding which is jointly optimized with learner hyperedge embedding from the hypergraph.

1) Representation Learning of Hypergraph: From the previous module we obtained the feature matrix of exercises $\widetilde{\mathbf{E}}$. And suppose there are sequences of p students answering exercises $l_i = (e_1, a_1), (e_2, e_2)...(e_t, a_t)$ to form $L_h = [l_1, l_2, ..., l_p]$ for constructing the HG. We set each hyperedge's weight to 1 for equality, but exercise nodes in hypergraph should have different importance. So we employ a dynamic incidence matrix attention score calculation inspired by [17] to align with the hypergraph with edge-dependent vertex weights [25]. For a vertex x_i and its hyperedge x_j the attentional score is presented as follows:

$$H_{ij} = \frac{\exp\left(\sigma\left(\text{sim}\left(x_i\mathbf{P}, x_j\mathbf{P}\right)\right)\right)}{\sum_{k\in\mathcal{N}_i}\exp\left(\sigma\left(\text{sim}\left(x_i\mathbf{P}, x_k\mathbf{P}\right)\right)\right)} \tag{13}$$

where $\text{sim}\left(x_i, x_j\right) = \mathbf{a}^\mathbf{T}\left[x_i, x_j\right]$ calculates the similarity value with a weight vector $\mathbf{a}^\mathbf{T}$, $\mathbf{P}$ is a learnable weight matrix.

Then the formula of the l-th HGNNConv layer that aggregates the vertices and hyperedges and performs $vertex$-$hyperedge$-$vertex$ propagation can be written as:

$$\mathbf{X}^l = \sigma\left(\mathbf{D}_v^{-1}\mathbf{H}\mathbf{W}_\epsilon\mathbf{B}_\epsilon^{-1}\mathbf{H}^\top\mathbf{X}^{l-1}\mathbf{P}^{l-1}\right) \tag{14}$$

where W_ϵ is initialized as an identity matrix for treating all hyperedges in a equal way. Here, we recall that D_v and B_ϵ are the degree matrices of the nodes and hyperedges in HG. Learner embeddings can be represented by aggregating the embeddings of exercises. And we can also extract the hyperedge feature in l-th layer as follows:

$$\mathbf{Y}^l = \mathbf{W}_\epsilon\mathbf{B}_e^{-1}\mathbf{H}^\top\mathbf{X}^{l-1} \tag{15}$$

After propagation using the multi-layer $HGNNConv$, we obtain the aggregate embedding of learners and exercises. That is, $\widetilde{\mathbf{L}}_\mathbf{h} = [\widetilde{\mathbf{l}}_1, \widetilde{\mathbf{l}}_2, ..., \widetilde{\mathbf{l}}_p]$ for all learners' representation.

2) Representation Learning of Line Hypergraph: Let's say $A \in \mathbb{R}^m$ is the adjacency matrix setting the entries $A_{pq} = w_{pq}$ according to (4). Let $\hat{A} = A + I$ where I is an identity matrix and $\hat{\mathbf{D}}$ is a diagonal matrix with $\hat{\mathbf{D}}_p = \sum_{q=1}^m \hat{\mathbf{A}}_{p,q}$. And the node features X are obtained from the averaging exercises embeddings contained in hyperedge. Then the line graph convolution in l-th layer is denoted as:

$$\mathbf{H}^l = \sigma(\hat{\mathbf{D}}^{-\frac{1}{2}}\hat{\mathbf{A}}\hat{\mathbf{D}}^{-\frac{1}{2}}\mathbf{X}^{l-1}\Theta^{l-1}) \tag{16}$$

Here, X is the input feature $\widetilde{\mathbf{L}}_h$ of learners from hypergraph, $\sigma()$ is a nonlinear activation function where we use LeakyRelu and Θ is the trainable weight parameters of GCN which transforms the matrix X to higher space.

So that we can apply the GCNs to the LHG, which takes the learners' embedding as input feature and outputs a new learners' embedding $\widetilde{\mathbf{L}}_l$ through multi-layer's propagation. Finally, we can obtain the final learners' embedding representation by concatenating $\widetilde{\mathbf{L}}_h$ and $\widetilde{\mathbf{L}}_l$, and mapping it to a lower dimensional embedding $\widetilde{\mathbf{L}}$.

Authorized licensed use limited to: HUNAN UNIVERSITY. Downloaded on December 04,2023 at 10:41:15 UTC from IEEE Xplore. Restrictions apply.

C. Knowledge Evolution Module

This module employs LSTM as a component for modeling temporal and sequential data. Different from GIKT [11], we follow [8] to take the difficulty of exercises and average response time along with exercise embedding as input to learn the hidden state.

1) Definition of Difficulty and Average Response Time: Because the difficulty of exercises and average response time are not explicitly provided in the dataset, here we define and calculate the exercise difficulty and average response time.

To exclude the impact of minor data discrepancies, exercises interacted with fewer than five students are excluded from the difficulty score calculation and are assigned a value of 0.5 directly. Take the set $E = \{e_0, e_1, ..., e_m\}$ with m exercises, and calculate the difficulty d_i of the exercise $e_i \in E(0 \leq i \leq m)$ by counting the responses of all students on that exercise:

$$d_i = \begin{cases} \frac{W_i}{N_i}, & \text{if } n_i \geq 5 \\ 0.5, & \text{else} \end{cases} \tag{17}$$

Here, n_i is the number of learners that interacted with the exercise e_i; N_i and W_i represent the total number of attempts and incorrect answers by all learners on this exercise e_i, respectively.

As a measure of the average response time, we employ the elapsed time T_i, which corresponds to the amount of time the student spent on the i-th exercise. Let a learner m's interaction sequence $l_m = \{(e_1, T_1), (e_2, T_2), ..., (e_t, T_t)\}$ at time step t. The average response time is calculated as follow:

$$art_t = \frac{\sum_{i=1}^{t} T_i}{t} \tag{18}$$

We subsequently use fully connected layer and concatenation to fuse d_i and art_t into a 100-dimensional vector $\mathbf{da}$:

$$\mathbf{da}_t = \mathbf{W}_f[\mathbf{d}_t, \mathbf{art}_t] + \mathbf{b}_f \tag{19}$$

2) Knowledge hidden state: In deep learning-based knowledge tracing, the knowledge state is a vector that is used to represent a learner's mastery of knowledge. Referring to [8], the exercise embedding, difficulty embedding and average response time features will be concatenated along with answer embedding to form the interaction embedding $\mathbf{I}_i$ at every time step i:

$$\mathbf{I}_t = \text{Relu}\left(\mathbf{W}_2\left(\left[\mathbf{W}_1\left[\mathbf{da}_t, \widetilde{\mathbf{e}}_\mathbf{t}, \mathbf{a_t}\right] + \mathbf{b}_1\right]\right) + \mathbf{b}_2\right) \tag{20}$$

where $\mathbf{W}_1, \mathbf{W}_2$ and $\mathbf{b}_1, \mathbf{b}_2$ are trainable matrices and parameters.

Then, LSTM [5] is used to learn the hidden state from interaction embedding:

$$\mathbf{i}_t = \sigma\left(\mathbf{W}_i\left[\mathbf{I}_t, \mathbf{h}_{t-1}, \mathbf{c}_{t-1}\right] + \mathbf{b}_i\right) \tag{21}$$

$$\mathbf{f}_t = \sigma\left(\mathbf{W}_f\left[\mathbf{I}_t, \mathbf{h}_{t-1}, \mathbf{c}_{t-1}\right] + \mathbf{b}_f\right) \tag{22}$$

$$\mathbf{o}_t = \sigma\left(\mathbf{W}_o\left[\mathbf{I}_t, \mathbf{h}_{t-1}, \mathbf{c}_{t-1}\right] + \mathbf{b}_o\right) \tag{23}$$

$$\mathbf{c}_t = \mathbf{f}_t\mathbf{c}_{t-1} + \mathbf{i}_t \tanh\left(\mathbf{W}_c\left[\mathbf{I}_t, \mathbf{h}_{t-1}\right] + \mathbf{b}_c\right), \tag{24}$$

$$\mathbf{h}_t = \mathbf{o}_t \tanh\left(\mathbf{c}_t\right) \tag{25}$$

where $\mathbf{c}_t$, $\mathbf{i}_t$, $\mathbf{f}_t$, and $\mathbf{o}_t$ represent the hidden state, cell state, input gate, forget gate and output gate respectively.

D. Prediction Module

By using the recap function, this module first draws attention to the useful historic interaction embedding and the learner embedding via the recap function [11], and then incorporates other relevant information to make predictions.

1) Recap the Important Information: This section addresses the extraction of two important pieces of information: the historical interaction embeddings $\mathbf{I}_i$ and the relevant learner embeddings $\mathbf{L}_i$. Since it is more difficult for LSTM to learn long-term dependencies in a long sequence, resulting in the current knowledge state h_t to potentially miss important information, the extraction of historical interaction embedding is intended to primarily address this issue. Furthermore, the relevant learners embeddings are those whose learning history, exercises and skills are most similar to those of the current student.

We use soft selection through an attention network for these two embeddings' extraction and then choose the top-k results with the highest attention scores, as mentioned in GIKT:

$$\mathbf{I}_a = \{\mathbf{I}_i \mid R_{i,t} \leq k, U_{i,t} \geq u, i \in [1, \ldots, t]\} \tag{26}$$

$$\mathbf{L}_a = \{\mathbf{L}_i \mid R_{i,m} \leq k, V_{i,m} \geq v, i \in [1, \ldots, m]\} \tag{27}$$

here R_{it} and R_{im} are the ranking of the attention function where we use cosine similarity, $U_{i,t}$ and $V_{i,m}$ are the attention coefficients. u and v are the threshold values of similarity used to eliminate less relevant embeddings.

2) Learner Performance Prediction: This subsection considers all the embedding representations learned from the previous sections as input to a generalized interaction component based on a bi-attention mechanism and finally outputs the predictions. As we can see from (28), exercise embedding is fused with difficulty and average response time. Similar to the practice of using a generalized interaction module [11], HGKT outputs the predicted probability of learners performance on new exercises at the next time step, which is calculated as follows:

$$\widetilde{\mathbf{e}}_{t+1}^{I} = \text{Relu}\left(\mathbf{W}\left(\left[\mathbf{da}_{t+1}, \widetilde{\mathbf{e}}_{\mathbf{t+1}}\right]\right) + \mathbf{b}\right) \tag{28}$$

$$\beta_{i,j} = \text{Softmax}_{i,j}\left(\mathbf{W}^T\left[\boldsymbol{f}_i, \boldsymbol{f}_j\right] + \mathbf{b}\right) \tag{29}$$

$$P_t = \sum_{\boldsymbol{f}_i \in \mathbf{I}_a \cup \mathbf{L}_a\{\mathbf{h}_t\}} \sum_{\boldsymbol{f}_j \in \tilde{\mathbf{N}}_{e_{t+1}} \cup \left\{\tilde{\mathbf{e}}_{t+1}^{I}\right\}} \beta_{i,j} g\left(\boldsymbol{f}_i, \boldsymbol{f}_j\right) \tag{30}$$

where $[,]$ is the concatenation operator, $\mathbf{W}^T$ is a transposed trainable weight matrix of feed-forward network, $\beta_{i,j}$ represents the attention correlation coefficient between $\mathbf{f}_i$ and $\mathbf{f}_j$, $\tilde{\mathbf{N}}_{e_t}$ represents the concepts embedding of the neighboring vertices of exercise e_t in WBG, $g()$ is a function implemented with inner product.

Finally, HGKT is trained using gradient descent with Adam optimizing the model parameters by minimizing the binary

Authorized licensed use limited to: HUNAN UNIVERSITY. Downloaded on December 04,2023 at 10:41:15 UTC from IEEE Xplore. Restrictions apply.

cross-entropy loss $\mathcal{L}$ between the predicted result p_t and the true label a_t as follows:

$$\mathcal{L} = -\sum_{t \in T} \left(a_t \log \left(p_t \right) + \left(1 - a_t \right) \log \left(1 - p_t \right) \right) \tag{31}$$

where T denotes the entire exercise answering sequence in the training set.

V. Experiments

In order to evaluate the predictive performance of the model HGKT proposed in this paper, this section first introduces the three publicly available datasets used for the experiments, the evaluation metrics, and the baseline models, and finally discusses and analyzes the performance of the model in relation to the experimental results.

A. Overview of Datasets

The experiments in this paper validate the effectiveness of the proposed HGKT model using three publicly available datasets commonly found in the domain of knowledge tracing: Assist09, Assist12, and EdNet. All three datasets are derived from time-series interactions produced by learners on the platform in real ITS; see Table I for more details of the datasets.

- **ASSIST09** was generated by 4,217 students on the ASSISTment learning platform during the 2009–2010 academic year and is one of the most commonly used datasets in the knowledge tracing domain, where there is a many-to-many relationship between exercises and knowledge concepts, with a single exercise containing up to four knowledge concepts. We refer to previous work [33] and perform data cleaning on its skill builder section.
- **ASSIT12** is derived from the same learning platform as ASSIT09, but with a much larger amount of data, and was collected in the 2012–2013 academic year. The original dataset consists of 46,674 student interactions, where each exercise contains only one knowledge concept and one knowledge concept may appear in different exercises.
- **EdNet** is a hierarchical dataset collected from the Santa learning platform, comprised of four sub-datasets that record the behaviors of various learners. It is the largest and most comprehensive dataset to our knowledge, with a many-to-many relationship between exercises and knowledge concepts. We utilized the learning records of 5,000 randomly picked learners from the sub-dataset KT-1.

TABLE I: Summary of Datasets statistics.

Items	ASSIST09	ASSIST12	EdNet
# records	282,619	2,709,436	653,761
# students	3852	27,485	5000
# exercises	17,737	53,065	11,655
# concepts	123	265	188
exercises per concept	173	200	142
concepts per exercise	1.197	1.000	2.303
attempts per exercise	16	51	57
attempts per concept	2743	10,224	8411
average sequence length	42.15	93.45	69.22

B. Baseline Methods

To further verify the effectiveness of the HGKT, several classical and excellent models have been selected as baselines for comparison. They are briefly outlined as follows:

- **BKT** [34] is a classical model often used as a baseline model in the field of knowledge tracing, using a Hidden Markov Model (HMM) with hidden variables to model students' knowledge.
- **DKT** [4] is the first work since the introduction of knowledge tracing in deep learning to model students' knowledge acquisition using LSTM, a variant of recurrent neural networks.
- **DKT+** [35] is a patched version of DKT with an improved loss function that solves the reconstruction and waviness problems of DKT.
- **DKVMN** [36] is a model based on memory augmented neural networks for knowledge tracing, employing a static matrix called "key" to store concepts and a dynamic matrix called "value" to store knowledge states, and simulating the learning process by designing read and write operations to update the matrix "value". Moreover, it can automatically discover the underlying concepts of exercises.
- **PEBG-DKT** [10] and **PEBG-DKVMN** [10] use the pre-training embeddings of each question as input for DKT and DKVMN separately. Question embeddings are obtained from question-skill bipartite graph, along with a product layer to fuse the side information includes question difficulty and three kinds of relations.
- **GIKT** [11] is a knowledge tracing model that combines the graph convolutional neural network (GCN) and the recurrent neural network (RNN) . It creates a bipartite graph with exercises and knowledge concepts as nodes, then uses GCN aggregation on the graph structure to obtain node representations that feed into the LSTM. The recap module and generalized interaction mechanism are then used to perform the prediction task.

C. Experiment Setup

For all our experiments, we split the training set, validation set, and test set on these three datasets in the ratio of 60%-20%-20% and set the maximum sequence length to 200, which is the former 200 records of each learner. In order to evaluate the model's performance, the area under the ROC curve (AUC) is used as the evaluation metric, which takes the value range of [0, 1] and can exclude the impact of imbalanced data. The result less than 0.5 means that a random guess is likely to be better than the model's prediction performance, and the result closer to 1 indicates that the prediction is more accurate. We perform five-fold cross-validation and calculated the mean values of AUC.

Authorized licensed use limited to: HUNAN UNIVERSITY. Downloaded on December 04,2023 at 10:41:15 UTC from IEEE Xplore. Restrictions apply.

For model training, the exercise and concept embedding are initialized randomly with $torch.nn.embedding$, set all embedding dimension such as exercise, concept, learner and hidden state to 100. And the Adam optimizer is used for training with the learning rate of 0.001, dropout rate of 0.6 and batch size of 64, number of epochs 200 with an early stopping strategy. We implement a three-layer NENN with node-level attention for weighted bipartite graph and build a two-layer HGNNConv to employ the powerful capacity of hypergraph and a two-layer LHCN on the line graph of hypergraph. A stacked LSTM with two hidden layers is utilized with memory cell sizes of 200 and 100, respectively.

D. Overall Performance

Table II shows the experimental results of AUC for all the compared models on the student performance prediction task. Obviously, we can observe that the proposed HGKT outperforms the other methods by at least 1.81% on all three datasets, which demonstrates its overall effectiveness. And the bipartite graph-based methods such as PEBG and GIKT have some advantages over all other methods. Thus, the embeddings aggregated from the bipartite graph, as well as other side information such as difficulty and other relationships between question and skill, have a considerable effect on DKT and DKVMN.

TABLE II: The AUC results on different datasets.

Model	ASSIST09	ASSIST12	EdNet
BKT	0.6571	0.6204	0.6027
DKT	0.7392	0.7123	0.6702
DKT+	0.7721	0.7431	0.6945
DKVMN	0.7545	0.7286	0.7011
PEBG+DKT	0.7801	0.7524	0.7544
PEBG+DKVMN	0.7810	0.7636	0.7520
GIKT	0.7783	0.7729	0.7534
HGKT	**0.7982**	**0.7956**	**0.7873**

E. Ablation Study

We conduct three ablation studies on HGKT. First we evaluate the layer number of NENN and HGNN separately and then explore the importance and effectiveness of the various components and methods integrated in our model. Here, we use HGKT-nW for using traditional GCNs instead of edge weight and NENN on bipartite graph; HGKT-nDIM for using the binary incidence matrix instead of attention based dynamic non-negative real values; HGKT-nART for removing the feature of historic average response time; HGKT-nD for removing the feature of exercises' difficulty; and HGKT-nL represents removing the embedding of learner, namely removing the all hypergraph-related learning components.

Fig. 3a and Fig. 3b show the reason why we use a three-layer NENN and a two-layer HGNNConv for graph representation learning. Then, as shown in Fig. 3c, the performance of HGKT-nW, HGKT-nART and HGKT-nIMU decline slightly on three datasets, but HGKT-nD and HGKT-nL drop a litter more than them.

Then we give a summary:

- Utilizing the edge feature and attention mechanism on the bipartite graph may have the power to improve the semantic representation learning between exercises and concepts.
- By excluding the learner embedding, the results drop a little more, which implying that measuring the learners' learning, mastery process and relationship is helpful for prediction.
- When it comes to the side information, the difficulty of exercises is more important than the average response time, which means difficulty can better optimize the LSTM's hidden vectors' expression of knowledge state. It may also imply that the average response time can represent the learner's mastery degree to some extent.
- For the incidence matrix in hypergraph, using normalized edge-independent vertex weights to reflects the importance of exercises may improve the modeling of learner's learning process and form a better learner embedding.

VI. Conclusion

In this paper, we propose a hypergraph-based knowledge tracing model with GIKT as backend. Our method tries to incorporate the edge feature into bipartite graph learning using node level attention based NENN, which helps obtain a better and more semantic representation of exercises and concepts. Then we build a hypergraph to model the learners' exercise answering process using HGNN with edge-independent vertex weights in incidence matrix and discover the high-order relationships among learners. Next, the hypergraph is transformed into a simple graph named line hypergraph, where GCN is used to aggregate and obtain the learners embeddings. Finally, in the knowledge state evolution module, two pieces of side information, i.e., difficulty and average response time are integrated into LSTM to enhance the learning of hidden states. And for the prediction module, attention is drawn to the useful and relevant historic information and predictions are made based on all the embedding from previous modules. We validate the advantages of HGKT on three real-world datasets compared to some existing representative methods in terms of AUC. The results indicate that our model can achieve better performance.

References

[1] Kelum AA Gamage, Dilani I Wijesuriya, Sakunthala Y Ekanayake, Allan EW Rennie, Chris G Lambert, and Nanda Gunawardhana. Online delivery of teaching and laboratory practices: Continuity of university programmes during covid-19 pandemic. *Education Sciences*, 10(10):291, 2020.

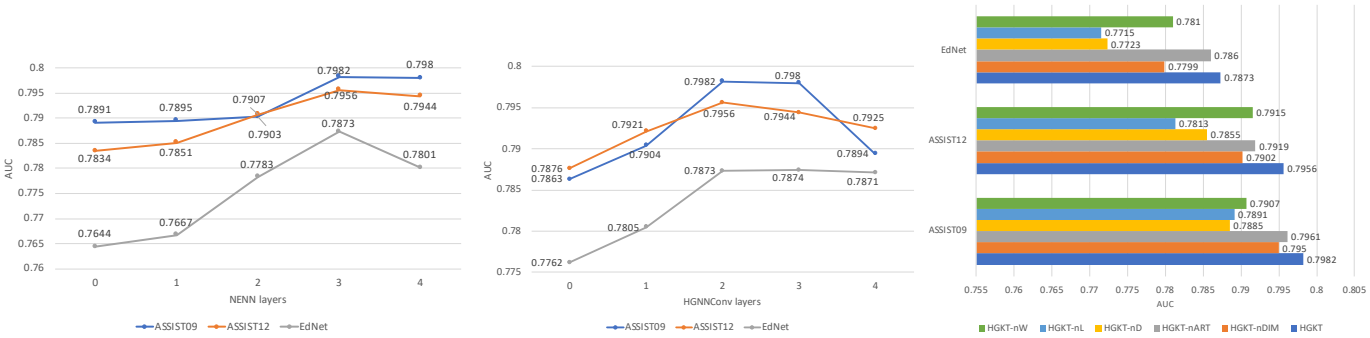
[2] Ching-Kun Hsu, Gwo-Jen Hwang, and Chih-Kai Chang. A personalized recommendation-based mobile learning approach to improving the reading performance of efl students. *Computers & Education*, 63:327–336, 2013.

[3] Ian Selwood and Rachel Pilkington. Teacher workload: using ict to release time to teach. *Educational Review*, 57(2):163–174, 2005.

[4] Chris Piech, Jonathan Bassen, Jonathan Huang, Surya Ganguli, Mehran Sahami, Leonidas J Guibas, and Jascha Sohl-Dickstein. Deep knowledge tracing. *Advances in neural information processing systems*, 28, 2015.

[5] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.

Authorized licensed use limited to: HUNAN UNIVERSITY. Downloaded on December 04,2023 at 10:41:15 UTC from IEEE Xplore. Restrictions apply.



(a) Effect of NENN layers.

(b) Effect of HGNNConv layers.

(c) Effect of the some other factors.

Fig. 3: Performance of ablation study under AUC.

[6] Renyu Zhu, Dongxiang Zhang, Chengcheng Han, Ming Gao, Xuesong Lu, Weining Qian, and Aoying Zhou. Programming knowledge tracing: A comprehensive dataset and a new model. *arXiv preprint arXiv:2112.08273*, 2021.

[7] Qi Liu, Zhenya Huang, Yu Yin, Enhong Chen, Hui Xiong, Yu Su, and Guoping Hu. Ekt: Exercise-aware knowledge tracing for student performance prediction. *IEEE Transactions on Knowledge and Data Engineering*, 33(1):100–115, 2019.

[8] Koki Nagatani, Qian Zhang, Masahiro Sato, Yan-Ying Chen, Francine Chen, and Tomoko Ohkuma. Augmenting knowledge tracing by considering forgetting behavior. In *The world wide web conference*, pages 3101–3107, 2019.

[9] Hiromi Nakagawa, Yusuke Iwasawa, and Yutaka Matsuo. Graph-based knowledge tracing: modeling student proficiency using graph neural network. In *2019 IEEE/WIC/ACM International Conference On Web Intelligence (WI)*, pages 156–163. IEEE, 2019.

[10] Yunfei Liu, Yang Yang, Xianyu Chen, Jian Shen, Haifeng Zhang, and Yong Yu. Improving knowledge tracing via pre-training question embeddings. *arXiv preprint arXiv:2012.05031*, 2020.

[11] Yang Yang, Jian Shen, Yanru Qu, Yunfei Liu, Kerong Wang, Yaoming Zhu, Weinan Zhang, and Yong Yu. Gikt: a graph-based interaction model for knowledge tracing. In *Machine Learning and Knowledge Discovery in Databases: European Conference, ECML PKDD 2020, Ghent, Belgium, September 14–18, 2020, Proceedings, Part I*, pages 299–315. Springer, 2021.

[12] Jiaying Meng, Ke Yu, and Xiaofei Wu. Enhancing graph neural networks with edge features through sequential representation. In *2020 IEEE 6th International Conference on Computer and Communications (ICCC)*, pages 2110–2114. IEEE, 2020.

[13] Yulei Yang and Dongsheng Li. Nenn: Incorporate node and edge features in graph neural networks. In *Asian conference on machine learning*, pages 593–608. PMLR, 2020.

[14] Claude Berge. *Graphs and hypergraphs*. 1973.

[15] Yifan Feng, Haoxuan You, Zizhao Zhang, Rongrong Ji, and Yue Gao. Hypergraph neural networks. In *Proceedings of the AAAI conference on artificial intelligence*, volume 33, pages 3558–3565, 2019.

[16] Mingzhi Chen, Quanlong Guan, Yizhou He, Zhenyu He, Liangda Fang, and Weiqi Luo. Knowledge tracing model with learning and forgetting behavior. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*, pages 3863–3867, 2022.

[17] Song Bai, Feihu Zhang, and Philip HS Torr. Hypergraph convolution and hypergraph attention. *Pattern Recognition*, 110:107637, 2021.

[18] Ziwei Zhang, Peng Cui, and Wenwu Zhu. Deep learning on graphs: A survey. *IEEE Transactions on Knowledge and Data Engineering*, 34(1):249–270, 2020.

[19] Marco Gori, Gabriele Monfardini, and Franco Scarselli. A new model for learning in graph domains. In *Proceedings. 2005 IEEE International Joint Conference on Neural Networks, 2005.*, volume 2, pages 729–734. IEEE, 2005.

[20] Thomas N Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907*, 2016.

[21] Will Hamilton, Zhitao Ying, and Jure Leskovec. Inductive representation learning on large graphs. *Advances in neural information processing systems*, 30, 2017.

[22] Xiangnan He, Kuan Deng, Xiang Wang, Yan Li, Yongdong Zhang, and Meng Wang. Lightgcn: Simplifying and powering graph convolution network for recommendation. In *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval*, pages 639–648, 2020.

[23] Zizhao Zhang, Yifan Feng, Shihui Ying, and Yue Gao. Deep hypergraph structure learning. *arXiv preprint arXiv:2208.12547*, 2022.

[24] Dengyong Zhou, Jiayuan Huang, and Bernhard Schölkopf. Learning with hypergraphs: Clustering, classification, and embedding. *Advances in neural information processing systems*, 19, 2006.

[25] Yue Gao, Yifan Feng, Shuyi Ji, and Rongrong Ji. Hgnn ˆ+: General hypergraph neural networks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2022.

[26] Sambaran Bandyopadhyay, Kishalay Das, and M Narasimha Murty. Line hypergraph convolution network: Applying graph convolution for hypergraphs. *arXiv preprint arXiv:2002.03392*, 2020.

[27] Hanshuang Tong, Zhen Wang, Yun Zhou, Shiwei Tong, Wenyuan Han, and Qi Liu. Hgkt: Introducing hierarchical exercise graph for knowledge tracing. *arXiv preprint arXiv:2006.16915*, 2020.

[28] Shiwei Tong, Qi Liu, Wei Huang, Zhenya Hunag, Enhong Chen, Chuanren Liu, Haiping Ma, and Shijin Wang. Structure-based knowledge tracing: An influence propagation view. In *2020 IEEE international conference on data mining (ICDM)*, pages 541–550. IEEE, 2020.

[29] Xiangyu Song, Jianxin Li, Yifu Tang, Taige Zhao, Yunliang Chen, and Ziyu Guan. Jkt: A joint graph convolutional network based deep knowledge tracing. *Information Sciences*, 580:510–523, 2021.

[30] Xiangyu Song, Jianxin Li, Qi Lei, Wei Zhao, Yunliang Chen, and Ajmal Mian. Bi-clkt: Bi-graph contrastive learning based knowledge tracing. *Knowledge-Based Systems*, 241:108274, 2022.

[31] Zhengyang Wu, Li Huang, Qionghao Huang, Changqin Huang, and Yong Tang. Sgkt: Session graph-based knowledge tracing for student performance prediction. *Expert Systems with Applications*, 206:117681, 2022.

[32] SGOPAL Patro and Kishore Kumar Sahu. Normalization: A preprocessing stage. *arXiv preprint arXiv:1503.06462*, 2015.

[33] Sein Minn, Feida Zhu, and Michel C Desmarais. Improving knowledge tracing model by integrating problem difficulty. In *2018 IEEE International conference on data mining workshops (ICDMW)*, pages 1505–1506. IEEE, 2018.

[34] Albert T. Corbett and John R. Anderson. Knowledge tracing: Modeling the acquisition of procedural knowledge. *User Modelling and User-Adapted Interaction*, 4(4):253–278, 1995.

[35] Chun-Kit Yeung and Dit-Yan Yeung. Addressing two problems in deep knowledge tracing via prediction-consistent regularization. In *Proceedings of the fifth annual ACM conference on learning at scale*, pages 1–10, 2018.

[36] Jiani Zhang, Xingjian Shi, Irwin King, and Dit-Yan Yeung. Dynamic key-value memory networks for knowledge tracing. In *Proceedings of the 26th international conference on World Wide Web*, pages 765–774, 2017.

Authorized licensed use limited to: HUNAN UNIVERSITY. Downloaded on December 04,2023 at 10:41:15 UTC from IEEE Xplore. Restrictions apply.